SQL
Window Functions
Interview Cheat Sheet

ROW_NUMBER | DENSE_RANK | NTILE | LAG / LEAD | FIRST/LAST VALUE | Running Total | Rolling Avg

14 Real Interview Questions • Clean SQL • Pro Tips

Q 1. Write a query to return only the most recent order per customer (by order_date). If two orders share the same date, keep the one with the higher order_id.

```
SELECT customer_id, order_id, order_date, total_amount
FROM (
    SELECT *,
        row_number() OVER(
            PARTITION BY customer_id
            ORDER BY order_date DESC, order_id DESC
        ) AS rn
    FROM orders
) x
WHERE x.rn = 1;
```

ROW_NUMBER() assigns a unique rank within each customer group ordered by date descending. The tiebreaker ORDER BY order_id DESC ensures that when two orders fall on the same date, the higher order_id wins. Filtering WHERE rn = 1 keeps exactly one row per customer.

Q 2. Find the top 3 products by total revenue (quantity x unit_price) within each product category. Include ties — if two products are tied for 3rd, both should appear.

```
WITH revenue AS (
    SELECT p.category, p.product_name,
           SUM(oi.quantity * oi.unit_price) AS total_revenue
    FROM products p
    JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY 1, 2
)
SELECT * FROM (
    SELECT *,
        DENSE_RANK() OVER(
            PARTITION BY category
            ORDER BY total_revenue DESC
        ) AS dr
    FROM revenue
) x
WHERE x.dr <= 3;
```

A CTE calculates each product's total revenue per category. DENSE_RANK() ranks products within each category without skipping numbers after ties. Two products tied at 3rd both get rank 3 and both pass the WHERE dr <= 3 filter. Use RANK() instead if you want gaps after ties.

Q 3. Write a query that shows each order for every customer alongside a running cumulative total of total_amount ordered by order_date. The running total should reset for each customer.

```
SELECT customer_id, order_date, total_amount,
    SUM(total_amount) OVER(
        PARTITION BY customer_id
        ORDER BY order_date
    ) AS running_total
FROM orders;
```

SUM() OVER with ORDER BY creates a running total — each row includes the sum of all previous rows in the same partition. PARTITION BY customer_id resets the cumulative sum at the start of each customer. The default frame (RANGE UNBOUNDED PRECEDING TO CURRENT ROW) handles this automatically.

Q 4. Calculate the total revenue per month. Then for each month, show the revenue from the previous month and the absolute change. Output: month, revenue, prev_month_revenue, mom_change.

```
WITH monthly_revenue AS (
    SELECT
        EXTRACT(MONTH FROM order_date) AS month,
        SUM(total_amount) AS revenue
    FROM orders
    GROUP BY month
    ORDER BY month
)
SELECT month, revenue,
    LAG(revenue) OVER (ORDER BY month) AS previous_month_revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change
FROM monthly_revenue;
```

The CTE aggregates revenue by month. LAG(revenue) looks back one row in the ordered result returning the prior month's revenue — no self-join needed. Subtracting gives the absolute month-over-month change. The first month returns NULL for previous_month_revenue since there is no preceding row.

Q 5. Flag customers whose next order came within 30 days.

```
SELECT customer_id, order_id, order_date,
    LEAD(order_date) OVER(
        PARTITION BY customer_id
        ORDER BY order_date
    ) AS next_order_date,
    CASE
        WHEN LEAD(order_date) OVER(
                PARTITION BY customer_id
                ORDER BY order_date
            ) - order_date <= 30
        THEN 'YES' ELSE 'NO'
    END AS is_repeated_within_30_days
FROM orders;
```

LEAD() looks forward to the next row within each customer's partition. Subtracting the current order_date from the next gives the gap in days. A CASE statement applies the rule: if gap is 30 days or less, flag as YES. The last order per customer returns NULL for next_order_date, which safely evaluates to NO.

Q 6. Compare each order to the customer's average order value.

```
SELECT customer_id, total_amount,
    ROUND(AVG(total_amount) OVER(
        PARTITION BY customer_id
    ), 2) AS average_value,
    total_amount - ROUND(AVG(total_amount) OVER(
        PARTITION BY customer_id
    ), 2) AS difference_in_amount
FROM orders
ORDER BY customer_id, order_date;
```

AVG() OVER(PARTITION BY customer_id) with no ORDER BY calculates the average across the entire customer partition — every row in the same group gets the same average value. Subtracting it from total_amount shows how far above or below average each order is. Positive difference means above-average spend.

Q 7. Calculate each customer's total lifetime spend. Use NTILE(4) to assign every customer a spending quartile (1 = lowest 25%, 4 = highest 25%). Label them: Q1, Q2, Q3, Q4.

```
WITH spend AS (
    SELECT customer_id, SUM(total_amount) AS total_spend
    FROM orders
    GROUP BY 1
)
SELECT customer_id, total_spend,
    NTILE(4) OVER(ORDER BY total_spend) AS quartile_num,
    CASE NTILE(4) OVER (ORDER BY total_spend)
        WHEN 1 THEN 'Q1 - Low'
        WHEN 2 THEN 'Q2 - Mid-Low'
        WHEN 3 THEN 'Q3 - Mid-High'
        WHEN 4 THEN 'Q4 - High'
    END AS segment
FROM spend;
```

NTILE(4) divides all customers into 4 equal-sized groups based on total_spend. Bucket 1 is the bottom 25% of spenders, bucket 4 is the top 25%. If customers don't divide evenly, earlier buckets get one extra row. The CASE statement converts the number into a readable label for marketing and CRM use.

Q 8. Calculate a 7-day rolling average of daily revenue.

```
WITH daily AS (
    SELECT order_date, SUM(total_amount) AS daily_revenue
    FROM orders
    GROUP BY order_date
)
SELECT order_date, daily_revenue,
    ROUND(
        AVG(daily_revenue) OVER(
            ORDER BY order_date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ), 2
    ) AS rolling_7_avg
FROM daily;
```

The frame clause ROWS BETWEEN 6 PRECEDING AND CURRENT ROW tells AVG() to look at the current row plus the 6 rows before it — exactly 7 days. Using ROWS (not RANGE) counts physical rows, so duplicate dates won't cause unexpected results. This smooths out daily spikes and makes revenue trends easier to read.

Q 9. Detect revenue drop of more than 20% versus previous month per sales rep.

```
WITH rep_revenue AS (
    SELECT o.rep_id, s.rep_name,
           EXTRACT(MONTH FROM o.order_date) AS month,
           SUM(o.total_amount) AS revenue
    FROM orders o
    LEFT JOIN sales_reps s ON o.rep_id = s.rep_id
    GROUP BY o.rep_id, s.rep_name, month
),
lagged AS (
    SELECT *,
        LAG(revenue) OVER(
            PARTITION BY rep_name
            ORDER BY month
        ) AS previous_revenue
    FROM rep_revenue
)
SELECT rep_name, month, revenue, previous_revenue,
    CASE
        WHEN previous_revenue IS NOT NULL
         AND (revenue - previous_revenue) * 1.0
             / previous_revenue < -0.20
        THEN 'drop>20%' ELSE 'OK'
    END AS flag
FROM lagged;
```

Two CTEs build the solution step by step. The first aggregates revenue per rep per month. The second uses LAG() to bring the previous month's revenue alongside each current row. The CASE applies the business rule: if revenue dropped more than 20%, flag it. Multiplying by 1.0 avoids integer division truncation.

Q 10. Show each order with the customer's very first and last order amount.

```
-- Simple version: first and last amounts only
WITH cte AS (
    SELECT customer_id, order_date, total_amount,
        FIRST_VALUE(total_amount) OVER(
            PARTITION BY customer_id
            ORDER BY order_date
        ) AS first_purchase,
        LAST_VALUE(total_amount) OVER(
            PARTITION BY customer_id
            ORDER BY order_date
            ROWS BETWEEN UNBOUNDED PRECEDING
                     AND UNBOUNDED FOLLOWING
        ) AS last_purchase
    FROM orders
)
SELECT DISTINCT customer_id, first_purchase, last_purchase
FROM cte;
```

```sql
-- Extended: include both dates and amounts
SELECT customer_id,
    MIN(first_order_date)   AS first_order_date,
    MIN(first_order_amount) AS first_order_amount,
    MIN(last_order_date)    AS last_order_date,
    MIN(last_order_amount)  AS last_order_amount
FROM (
    SELECT customer_id,
        FIRST_VALUE(order_date) OVER(
            PARTITION BY customer_id ORDER BY order_date
        ) AS first_order_date,
        FIRST_VALUE(total_amount) OVER(
            PARTITION BY customer_id ORDER BY order_date
        ) AS first_order_amount,
        LAST_VALUE(order_date) OVER(
            PARTITION BY customer_id ORDER BY order_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
        ) AS last_order_date,
        LAST_VALUE(total_amount) OVER(
            PARTITION BY customer_id ORDER BY order_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
        ) AS last_order_amount
    FROM orders
) t
GROUP BY customer_id;
```

FIRST_VALUE() works correctly with the default frame. LAST_VALUE() however defaults to RANGE UNBOUNDED PRECEDING TO CURRENT ROW — without changing the frame, it only looks up to the current row and gives wrong results. Always add ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING with LAST_VALUE.

Q 11. Find the 2nd highest order value per customer.

```sql
SELECT customer_id, order_id, order_date, total_amount
FROM (
    SELECT *,
        ROW_NUMBER() OVER(
            PARTITION BY customer_id
            ORDER BY total_amount DESC
        ) AS rn
    FROM orders
) x
WHERE x.rn = 2;
```

ROW_NUMBER() ordered by total_amount DESC gives rank 1 to the highest order per customer. Filtering WHERE rn = 2 returns the second-highest. This scales to any N — just change the WHERE clause. Note: ROW_NUMBER breaks ties arbitrarily. Use DENSE_RANK if you want ties to share a rank.

Q 12. Identify customers who placed orders in 3 consecutive months.

```
WITH order_month AS (
    SELECT customer_id,
           EXTRACT(MONTH FROM order_date) AS month,
           ROW_NUMBER() OVER(
               PARTITION BY customer_id
               ORDER BY EXTRACT(MONTH FROM order_date)
                   ) AS row_num
    FROM orders
),
consecutive_month AS (
    SELECT customer_id,
           (month - row_num) AS diff
    FROM order_month
)
SELECT customer_id
FROM consecutive_month
GROUP BY customer_id, diff
HAVING COUNT(*) >= 3;
```

The classic consecutive-rows trick: for truly consecutive months, (month - row_num) produces the same constant value. For example months 3, 4, 5 with row numbers 1, 2, 3 all give diff = 2. Grouping by customer_id and diff, then checking HAVING COUNT(*) >= 3, finds customers with at least 3 consecutive months. Works for any N.

Q 13. Calculate what % of total revenue each product contributes (revenue share).

```
WITH product_revenue AS (
    SELECT p.product_name,
           SUM(oi.quantity * oi.unit_price) AS total_revenue
    FROM products p
    JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY p.product_name
)
SELECT product_name,
       total_revenue,
       ROUND(
           (total_revenue * 100.0)
           / SUM(total_revenue) OVER(), 2
       ) AS revenue_pct
FROM product_revenue
ORDER BY revenue_pct DESC;
```

SUM(total_revenue) OVER() with empty parentheses calculates a grand total across the entire result set — no partition, no order. Dividing each product's revenue by this grand total and multiplying by 100 gives the percentage share. Multiplying by 100.0 avoids integer division. Much cleaner than a subquery or cross join.

Q 14. Find customers whose last order is their highest order. Also calculate a 3-order moving average revenue per sales rep.

```sql
-- Part 1: Customers whose last order = highest order
WITH ranked AS (
    SELECT customer_id, total_amount,
        ROW_NUMBER() OVER(
            PARTITION BY customer_id
            ORDER BY order_date DESC
        ) AS last_date,
        MAX(total_amount) OVER(
            PARTITION BY customer_id
        ) AS max_revenue
    FROM orders
)
SELECT customer_id
FROM ranked
WHERE last_date = 1
  AND total_amount = max_revenue;

-- Part 2: 3-order moving average per sales rep
SELECT rep_id, total_amount, order_date,
    ROUND(
        AVG(total_amount) OVER(
            PARTITION BY rep_id
            ORDER BY order_date
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
        ), 2
    ) AS moving_avg_3
FROM orders;
```

Part 1 combines two window functions: ROW_NUMBER() tags the most recent order (last_date = 1) and MAX() OVER finds each customer's all-time highest amount. Rows where both match identify customers with an improving spend trend. Part 2 uses ROWS BETWEEN 2 PRECEDING AND CURRENT ROW to average exactly 3 orders at a time per sales rep.

#SQL #WindowFunctions #DataAnalyst #InterviewPrep